

# DESAFIO PRÁTICO — NLP APLICADO

---

## Ouvidoria Inteligente: Triagem Semântica de Manifestações Cidadãs

**Disciplina:** Processamento de Linguagem Natural / Engenharia de IA

**Tema:** Representações Vetoriais, Busca Semântica e Chunking

**Prazo sugerido:** 2 semanas

**Modalidade:** Individual ou em duplas

### 1. Contexto do Caso

A Ouvidoria Geral de um município de médio porte recebe, em média, 4.000 manifestações por mês de cidadãos por meio de um formulário online. As manifestações são textos livres descrevendo problemas como buracos em vias públicas, falta de iluminação, demora no atendimento de saúde, ruído urbano, entre outros.

Atualmente, o sistema de triagem utiliza busca por palavras-chave (ex: "buraco", "luz", "saúde"). Isso gera três problemas graves:

- Duplicatas não detectadas: "asfalto esburacado" e "rua com buraco" são tratadas como diferentes;
- Temas relacionados não são agrupados: reclamações sobre "falta de remédio no posto" e "demora na consulta" deveriam compor um mesmo cluster de "saúde pública";
- Manifestações longas (com mais de 500 palavras) perdem contexto ao serem indexadas como um único bloco.

Sua missão, como engenheiro(a) de IA contratado(a) pela prefeitura, é construir um protótipo de sistema de triagem semântica que resolva esses problemas usando representações vetoriais modernas.

### 2. Base de Dados Fornecida

Você receberá um arquivo JSON chamado `manifestacoes.json` contendo 40 manifestações anônimas, cada uma com os campos:

| Campo                          | Descrição                         |
|--------------------------------|-----------------------------------|
| <code>id</code>                | identificador único (M001 a M040) |
| <code>data</code>              | data da manifestação (AAAA-MM-DD) |
| <code>categoria_oficial</code> | rótulo atribuído manualmente:     |

|              |                                                                       |
|--------------|-----------------------------------------------------------------------|
|              | "infraestrutura", "saúde", "segurança", "educação" ou "meio ambiente" |
| <b>texto</b> | descrição livre do cidadão (entre 50 e 800 caracteres)                |

🔗 Observação: cerca de 15% das manifestações são duplicatas semânticas (mesmo problema descrito com palavras diferentes) e 5 manifestações possuem textos longos (>500 caracteres) que devem ser trabalhados com chunking.

### 3. Objetivos de Aprendizagem

1. Comparar representações esparsas (BoW, TF-IDF) com representações densas (embeddings) em um cenário real;
2. Implementar busca semântica vetorial com similaridade de cosseno;
3. Aplicar técnicas de chunking para tratar documentos longos;
4. Detectar automaticamente manifestações duplicadas;
5. Visualizar o espaço semântico com PCA e t-SNE;
6. Construir uma interface interativa com Streamlit.

### 4. Entregas Obrigatórias

#### Entrega 1 — Análise Comparativa de Representações (2,5 pts)

Utilizando o corpus das 40 manifestações, gere representações BoW, TF-IDF e Embeddings (modelo à sua escolha, preferencialmente multilíngue como paraphrase-multilingual-MiniLM-L12-v2 ou BAAI/bge-small-pt-v1.5). Para os pares abaixo, calcule a similaridade de cosseno em cada representação e discuta os resultados:

- M003 ("buraco enorme na Av. Brasil") × M017 ("asfalto todo esburacado da avenida principal");
- M008 ("posto de saúde sem médico") × M022 ("falta atendimento no PSF");
- M008 ("posto de saúde sem médico") × M031 ("lâmpada queimada na praça").

Entregue um notebook (.ipynb) com o código, as tabelas comparativas e um parágrafo conclusivo sobre as limitações de cada abordagem.

#### Entrega 2 — Detecção de Duplicatas (2,5 pts)

Construa uma função `detectar_duplicatas(textos, limiar=0.85)` que receba a lista de manifestações e retorne todos os pares com similaridade acima do limiar. Justifique a escolha do limiar e apresente:

- A matriz de similaridade completa (heatmap);
- A lista de pares duplicados encontrados;
- Uma análise de falsos positivos e falsos negativos (compare com as duplicatas reais do dataset).

### Entrega 3 — Chunking de Manifestações Longas (2,0 pts)

As 5 manifestações mais longas devem ser divididas em chunks usando a estratégia RecursiveCharacterTextSplitter do LangChain. Teste pelo menos duas configurações diferentes de (chunk\_size, chunk\_overlap) e responda:

- Como o overlap influencia a coesão semântica entre chunks consecutivos?
- Qual configuração preserva melhor o sentido das denúncias? Justifique com exemplos.
- Visualize os chunks no espaço 2D (PCA ou t-SNE). Os chunks de uma mesma manifestação ficam próximos?

### Entrega 4 — Buscador Semântico + App Streamlit (3,0 pts)

Desenvolva uma aplicação Streamlit (app\_ouvidoria.py) com as seguintes abas:

- 🔍 Busca Semântica: o cidadão digita uma descrição livre e o sistema retorna as 5 manifestações mais similares, com score e destaque por cor ( $\text{score} > 0.7$ ,  $\text{score} > 0.5$ , demais);
- 📊 Base Completa: tabela com todas as manifestações e botão para gerar a matriz de similaridade;
- 📍 Espaço Vetorial: visualização 2D (PCA/t-SNE selecionável) das manifestações, coloridas por categoria oficial — o aluno deve comentar se os clusters semânticos coincidem com as categorias;
- 🗂️ Chunking: área para colar uma manifestação longa, escolher estratégia e parâmetros, e visualizar os chunks gerados + seus embeddings.

O app deve ser executável com streamlit run app\_ouvidoria.py e conter sidebar com seleção de modelo de embedding e top-k configurável.

## 5. Critérios de Avaliação

| Critério             | Peso | O que será avaliado                                     |
|----------------------|------|---------------------------------------------------------|
| Correção técnica     | 30%  | Código funcional, similaridades calculadas corretamente |
| Qualidade da análise | 25%  | Discussões fundamentadas, comparações bem argumentadas  |
| Criatividade no app  | 15%  | Interface intuitiva, visualizações claras, UX cuidada   |
| Organização          | 10%  | Notebook limpo, comentários, estrutura lógica           |
| Entrega completa     | 20%  | Todos os arquivos entregues no prazo                    |

## 6. Arquivos a Entregar

- análise\_comparativa.ipynb — Entrega 1;

- `deteccao_duplicatas.ipynb` — Entrega 2;
- `chunking_manifestacoes.ipynb` — Entrega 3;
- `app_ouvidoria.py` — Entrega 4;
- `RELATORIO.pdf` — documento de até 5 páginas sintetizando decisões, dificuldades e aprendizados.

## 7. Dicas e Recursos

- Use modelos multilíngues para melhor captura do português brasileiro;
- Explore a biblioteca `sentence-transformers` e compare pelo menos dois modelos;
- No Streamlit, use `st.cache_resource` para o modelo e `st.cache_data` para os embeddings;
- Considere aplicar um `threshold` dinâmico para duplicatas (ex: percentil 90 da distribuição de similaridades);
- Documente tudo: o código será lido por outra pessoa da equipe.

## 8. Referências

- Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.
- Documentação `sentence-transformers`: <https://www.sbert.ai/>
- Documentação LangChain Text Splitters: [https://python.langchain.com/docs/concepts/text\\_splitters/](https://python.langchain.com/docs/concepts/text_splitters/)
- Materiais de apoio: `lab_representacoes.ipynb`, `laborat_ecommerce_produtos.ipynb`, `app_faq_semantico.py`, `app_chunks_embeddings.py`.

**Bom trabalho! 🍀**